Spring Boot - Cheat Sheet

Contents

Spring Boot is an opinionated layer on top of the Spring Framework. It picks sensible defaults, wires up beans it detects on the classpath, and embeds a web server, so a service starts from a single `main` method with no XML. You add a starter dependency, annotate one class, and you have a running application.

Three ideas carry the whole framework: starters bring in curated dependency sets, auto-configuration wires beans based on what is on the classpath, and an embedded server means the jar runs itself. Everything else is Spring underneath.

1. The three ideas

Idea	What it does
Starters	`spring-boot-starter-*` pulls a curated, version-aligned set of dependencies
Auto-configuration	Beans are configured automatically from classpath and properties, and back off the moment you define your own
Embedded server	Tomcat (or Netty, Jetty) is packaged inside the jar, so `java -jar app.jar` is the whole deployment

Auto-configuration is conditional. See a `DataSource` on the classpath and a `spring.datasource.url`, and Boot configures one. Define your own `DataSource` bean and Boot steps aside. This “configure unless the developer already did” rule is the core of how it stays out of your way.

2. Application entry point

```java
@SpringBootApplication
public class OrdersApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrdersApplication.class, args);
    }
}
```

`@SpringBootApplication` is three annotations in one:

- `@Configuration`: this class can declare `@Bean` methods.
- `@EnableAutoConfiguration`: turn on auto-configuration.
- `@ComponentScan`: scan this package and everything below it for beans.

> **Trap**
>
> That last one is a frequent trap. Component scanning starts at the main class's package and goes down. A bean in a sibling package the scan never reaches simply does not exist, and Spring reports a missing dependency instead of a scanning problem.

3. Beans and dependency injection

Spring builds objects and injects their dependencies. You mark a class as a bean, and Spring manages its lifecycle.

Annotation	Meaning
`@Component`	A generic Spring-managed bean
`@Service`	A `@Component` for business logic
`@Repository`	A `@Component` for data access, adds exception translation
`@RestController`	A `@Component` that serves HTTP and returns response bodies
`@Configuration` + `@Bean`	Declare beans explicitly in a method

Prefer constructor injection. It makes dependencies required and final, keeps the class testable without Spring, and surfaces circular dependencies at startup instead of hiding them.

```java
@Service
public class OrderService {
    private final OrderRepository repository;
    private final PaymentClient payments;

    // single constructor, no @Autowired needed
    public OrderService(OrderRepository repository, PaymentClient payments) {
        this.repository = repository;
        this.payments = payments;
    }
}
```

Declare a bean explicitly when the type is not yours to annotate:

```java
@Configuration
public class ClientConfig {
    @Bean
    public PaymentClient paymentClient(@Value("${payments.url}") String url) {
        return new PaymentClient(url);
    }
}
```

4. Configuration and profiles

Configuration lives in `application.yml` or `application.properties`, outside the code.

```yaml
server:
  port: 8080
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/orders
payments:
  url: https://payments.internal
  timeout-ms: 2000
```

Read it three ways:

- `@Value("${payments.url}")` for a single value.
- `@ConfigurationProperties(prefix = "payments")` to bind a whole group to a typed POJO. This is the one to reach for.
- Inject `Environment` and call `getProperty` for dynamic lookups.

```java
@ConfigurationProperties(prefix = "payments")
public record PaymentProperties(String url, int timeoutMs) {}
```

Profiles

A profile is a named set of config for an environment. Put environment overrides in `application-{profile}.yml` and activate one:

```
java -jar app.jar --spring.profiles.active=prod
```

`@Profile("prod")` on a bean includes it only under that profile. Property precedence, highest first: command-line args, then environment variables, then `application-{profile}.yml`, then `application.yml`. Higher sources win, so a `SERVER_PORT` env var overrides the file.

5. Web layer

```
@RestController
@RequestMapping("/orders")
public class OrderController {
    private final OrderService service;

    public OrderController(OrderService service) {
        this.service = service;
    }

    @GetMapping("/{id}")
    public Order get(@PathVariable long id) {
        return service.findById(id);
    }

    @PostMapping
    public ResponseEntity<Order> create(@Valid @RequestBody CreateOrder body) {
        Order created = service.create(body);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }
}
```

- `@PathVariable` binds a URL segment, `@RequestParam` a query parameter, `@RequestBody` the deserialized JSON body.
- `@Valid` triggers bean validation on the body and returns 400 on failure.
- Return `ResponseEntity` when you need to set the status or headers, or the object directly for a plain 200.

6. Error handling

Centralize it with `@RestControllerAdvice` instead of try-catch in every handler.

```java
@RestControllerAdvice
public class ApiExceptionHandler {
    @ExceptionHandler(OrderNotFoundException.class)
    public ResponseEntity<ApiError> handle(OrderNotFoundException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body(new ApiError(e.getMessage()));
    }
}
```

7. Data access with Spring Data JPA

Declare a repository interface. Spring generates the implementation.

```java
@Entity
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String status;
    // getters, setters
}

public interface OrderRepository extends JpaRepository<Order, Long> {
    List<Order> findByStatus(String status);   // derived from the method name

    @Query("select o from Order o where o.status = :s and o.total > :min")
    List<Order> findExpensive(String s, BigDecimal min);
}
```

- `JpaRepository` gives you `save`, `findById`, `findAll`, `delete`, and paging for free.
- Method names like `findByStatus` are parsed into queries. `@Query` takes over when the name would get unwieldy.
- Put `@Transactional` on the service method that spans several writes. Reads can be `@Transactional(readOnly = true)`.

8. Actuator

`spring-boot-starter-actuator` exposes operational endpoints. They are off the web by default; opt in.

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
```

- `/actuator/health` for liveness and readiness, used by Kubernetes probes.

- `/actuator/metrics` and, with `micrometer-registry-prometheus`, `/actuator/prometheus` for scraping.
- Micrometer is the metrics facade, so the same code exports to Prometheus, Datadog, or others by swapping the registry.

9. Testing

Boot ships test slices so you load only the part of the context you are testing.

Annotation	Loads	Use for
`@SpringBootTest`	The full application context	End-to-end and integration tests
`@WebMvcTest`	The web layer only, with MockMvc	Controller tests
`@DataJpaTest`	JPA and an in-memory database	Repository tests

```
@WebMvcTest(OrderController.class)
class OrderControllerTest {
    @Autowired MockMvc mvc;
    @MockitoBean OrderService service;          // replaces the real bean with a mock

    @Test
    void returnsOrder() throws Exception {
        when(service.findById(1L)).thenReturn(new Order(1L, "PAID"));
        mvc.perform(get("/orders/1"))
           .andExpect(status().isOk())
           .andExpect(jsonPath("$.status").value("PAID"));
    }
}
```

`@MockitoBean` replaced `@MockBean` in Spring Boot 3.4. It swaps a bean in the context for a Mockito mock.

10. Packaging and running

The `spring-boot-maven-plugin` builds an executable fat jar with every dependency and the embedded server inside.

```
mvn spring-boot:run                          # run in place during development
mvn clean package                            # build the fat jar
java -jar target/orders-1.0.0.jar            # run it anywhere with a JVM
```

The same jar is what goes into a Docker image, usually on a slim JRE base.

11. Best practices

- **Use constructor injection.** Required, final, testable, and circular dependencies fail loudly at startup.

- **Bind config with `@ConfigurationProperties`.** Typed, validated, grouped. Scatter `@Value` only for one-offs.
- **Keep the main class at the root package.** Everything below it is scanned. Beans above or beside it are not.
- **Let auto-configuration work, override deliberately.** Define your own bean only when the default is wrong, and know it disables the default.
- **Put `@Transactional` on the service, not the controller or repository.** The transaction boundary belongs to the business operation.
- **Externalize every environment difference.** Profiles and environment variables, never a branch on a hardcoded flag.
- **Expose only the actuator endpoints you use.** The defaults are conservative for a reason.
- **Return `ResponseEntity` for anything but a plain 200.** Status and headers should be explicit.

12. Common gotchas

- **Field injection hides problems.** `@Autowired` on a field is untestable without Spring and lets circular dependencies slip through. Use the constructor.
- **Beans outside the scanned package vanish.** A bean in a package the component scan never reaches is invisible, and the error points at the consumer, not the cause.
- **`@Transactional` self-invocation does nothing.** Calling one `@Transactional` method from another method in the same bean bypasses the proxy, so no transaction starts. Split them across beans.
- **The N+1 query trap.** A lazy association loaded in a loop fires one query per row. Use a join fetch or an entity graph.
- **Two beans of the same type.** Spring cannot choose without `@Primary` or `@Qualifier`, and startup fails.
- **`application.properties` and `application.yml` both present.** Properties usually win and the YAML looks ignored. Pick one format.
- **Auto-configuration silently backs off.** Define a bean Boot also configures and its default disappears. Handy, but surprising when unintended.
- **Actuator exposure is opt-in.** A missing `/actuator/metrics` is usually just an endpoint not added to `exposure.include`.
- **Spring Boot 3 moved to `jakarta.*`.** Old `javax.*` imports will not compile. Migrating from Boot 2 means changing every one.
- **The fat jar is not explodable like a normal jar.** Its layout is Boot-specific. Unpack it with Boot's tooling or layered Docker builds, not a plain `unzip` into a classpath.

13. Quick reference

Concept	Key fact
`@SpringBootApplication`	`@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`

Concept	Key fact
Starter	Curated dependency bundle, `spring-boot-starter-*`
Auto-configuration	Wires beans from classpath, backs off when you define your own
`@Component` family	`@Service`, `@Repository`, `@RestController`
Injection	Constructor injection, no `@Autowired` needed for one constructor
`@Bean`	Declare a bean explicitly inside `@Configuration`
`@Value`	Inject a single property
`@ConfigurationProperties`	Bind a group of properties to a typed object
Profile	Per-environment config, `application-{profile}.yml`
Precedence	CLI args over env vars over profile file over base file
`@RestController`	HTTP handler returning serialized bodies
`@RestControllerAdvice`	Central exception handling
`JpaRepository`	CRUD, paging, derived queries for free
`@Transactional`	Transaction boundary, put it on the service
Actuator	Health, metrics, Prometheus, opt-in via exposure
Test slices	`@SpringBootTest`, `@WebMvcTest`, `@DataJpaTest`
`@MockitoBean`	Replaces a context bean with a mock (was `@MockBean`)
Packaging	Executable fat jar via `spring-boot-maven-plugin`